

ТЕМА
СТРУКТУРИРАНИ/СЪСТАВНИ ДАННИ
АТД СТРУКТУРА/ЗАПИС

ABSTRACT DATA TYPE (АТД)

- АТД е математически модел на тип данни, който (тип) се дефинира семантично от гледна точка на потребителя.
- Те имат собствено множество от стойности и набор от операции, които могат да бъдат прилагани върху тях.
- Теоретична единица използвана за абстрактно описание на алгоритми.
- Имплементира се с конкретна структура от данни (data structures).
- АТД се изграждат чрез един или няколко от простите типове данни (primitive data types).
- Програмистът решава какви операции са необходими за съответния АТД и много често ги реализира чрез написването на съответния код.
- АТД имат голяма роля при обектно ориентираното програмиране.
- Примери: запис, списък, стек опашка, дърво и др.

УВОД

- Компютърните програми + КС заменят ръчните човешки дейности по обработка на някакви данни.
- Тези данни обикновено са „**сложни**“ - големи по обем и логически свързани, структурирани по някакъв начин и различни по характер.
- Да вземем например склад. Ако се обработват данни, ръчно, ще бъде необходимо да се поддържа каталог от артикули в склада, като **за всеки** артикул ще трябва да съхраняваме информация за: наименование, производител, ед. цена, брой в склада, дата производство, категория и мн. други
- При компютърната обработка на данни е необходимо да се осигури възможност за подобен подход при дефиниране и обработката.
- С други думи е необходимо да се предоставя възможност да се работи със структурирани данни, които са от **различен тип**(числа, стрингове, булеви, масиви и др.)
- Използват се специален тип **променливи**, които могат да съдържат данни от различни типове, наричат се още **записи**

АТД СТРУКТУРА

Структурата (запис, декартово произведение, *structure*, *record*, *cartesian product*) е съставен тип, представляващ **подредена крайна съвкупност от елементи, които могат да се различават по своя тип**. До компонентите е осигурен **директен достъп**

- **Основни понятия и характеристики:**
 - Компонентите се наричат **полета** или **членове** (*fields*, *members*) на структурата
 - Записът е един от малкото **комбинирувани АТД** (т.е. чиито компоненти от своя страна могат да са от различни АТД)
- **Структурата дава възможност няколко променливи от различен тип да бъдат обединени в една единица**

АТД СТРУКТУРА – ПРИЛОЖЕНИЕ

- За моделирането на обекти, които имат много на брой характеристики (и за обръщение към този обект само с едно име) – служи за групиране на смислово свързани данни, които в общия случай представляват характеристики на едно понятие

Пример: Понятието правилен многоъгълник се описва чрез две характеристики – брой на страните и дължина на страната

- За дефинирането на други структури от данни

Пример: В банките клиентите се описват чрез няколко признака (например име, адрес, номер на сметка, ЕГН), някои от които на свой ред могат съдържат също няколко признака (например адресът се състои от улица, номер, град, окръг и пощенски код). Такива множества от данни могат да бъдат моделирани чрез записите.

АТД СТРУКТУРА: ДОСТЪП И КОНСТРУИРАНЕ

- **Достъп:**

Директният достъп до компонентите се осъществява чрез съпоставените им уникални **имена**, а не чрез позицията им (както е при масивите)

- **Конструиране** – стойността на структурата може да се конструира по два начина:

- **чрез последователно изброяване** на стойностите на всички полета на записа, като се ограждат в “скоби” **{ }**
- **чрез достъп до всички полета** на записа за задаване на стойностите им

СТРУКТУРИТЕ В ЕЗИЦИТЕ ЗА ПРОГРАМИРАНЕ

- **Типични характеристики** в ЕП от високо ниво:
 - записите са стандартен съставен тип за много от езиците, но не всички (в Java, PHP не съществуват структури)
 - стойностите им се разглеждат като съставени от няколко именувани полета

- **Дефиниране** – декартовото произведение на `INTEGER` $\times$ `REAL` (Pascal) :

```
TYPE Polygon =  
    RECORD  
        NoOfSides : INTEGER;  
        SizeOfSides : REAL  
    END;  
VAR poly1, bigsquare : Polygon;
```

- **Достъп** – Достъпът до поле на запис се осъществява чрез т.нар. “точково” означаване (*dot notation*):

```
circumference := poly1.NoOfSides * poly1.SizeOfSides;
```

АТД СТРУКТУРА В C++: ФМ И ЗМ

- **Физически модел**

В паметта на компютъра всяка величина от тип структура (запис) заема толкова място, колкото е необходимо, за да бъдат съхранени всички нейни компоненти

- **Знаков модел**

В C++ структурите служат не само като модел на декартово произведение на различни множества от стойности, те всъщност са специален вид класове.

- **Синтаксис** за деклариране на тип структура и променливи от този тип:

```
struct <име>незад { <декл. на полета>незад } <декл. на променливи>незад;
```

, където:

- <име> е името на типа-структура
- <декл. на полета> е списък от типове и имена на променливи, разделени с “.”, които представляват **полетата** на структурата (**членове-данни**). Полетата могат да бъдат от всеки тип освен `void`, непълен тип (затова не могат да бъдат от типа на структурата, която се декларира в момента, но могат да бъдат указател към нея) или функция
- <декл. на променливи> е списък от имена на променливи от този тип структура.

Препоръчва се **променливите от даден тип структура** да се декларират отделно, като се спазва следния синтаксис:

```
<име на структура> <име на променлива>;
```

АТД СТРУКТУРА В C++: ЗМ – ПРИМЕРИ ЗА ДЕКЛАРИРАНЕ

(1)

```
struct Point {  
    int x;  
    int y;  
};  
Point p1,p2;
```

(2)

```
struct Child {  
    double height;  
    double weight;  
    int years;  
    int months;  
    char gender;  
} Ivan, Maria;
```

(3)

```
struct BookInfo  
{  
    string title;  
    string author;  
    string publisher;  
    double price;  
};  
...  
BookInfo bookList[20];  
// масив от структури
```

(4) Вложена структура

```
struct Date { //дата  
    int Day;  
    int Mon;  
    int Year;  
};  
enum Merka{Kg, M, L, Br} ;  
struct Stoka { //стока  
    char Ime[30];  
    Merka Mer;  
    double EdCena;  
    Date GodnaDo; //вложен  
                //запис  
} S;
```

(5)

```
struct movies {  
    string title;  
    int year;  
};  
  
movies amovie;  
movies * pmovie; // указател
```

АТД СТРУКТУРА В C++: ЗМ – МНОЖЕСТВО ОТ СТОЙНОСТИ

Множеството от стойности на структурата е:

Съвкупността от всички възможни комбинации от стойности на нейните полета. Стойността на всяка променлива от даден тип структура съдържа всички декларирани полета

АТД СТРУКТУРА В C++: ОПЕРАЦИЯ СЕЛЕКТОР

Изразът за достъп до членове (*member-selection expression*) на структура има две форми:

- Универсална — чрез “точково” означаване:
 - за променливи-структури или псевдоними на структури

```
p1.x = 150; p1.y = 50;
Point p01, &pps=p01;
pps.x = 150;
```
 - за достъп до структури, към които сочи указател

```
Point *pp;
//...
(*pp).x = 150;
(*pp).y = 50;
```
- Само за достъп до структури, към които сочи указател — чрез “->”:

```
int a = pp->x; //еквивалентно на a = (*pp).x;
cout << pS->GodnaDo.Day;
```

АТД СТРУКТУРА В C++: ЗМ – КОНСТРУКТОР И ИНИЦИАЛИЗИРАНЕ

1. Поелементно (чрез обхождане)

```
Date toDate; Stoka S;
```

```
//конструира toDate
```

```
toDate.Day=30; toDate.Mon=1; toDate.Year=2018;
```

```
//конструира S
```

```
strcpy(S.Ime, "Ivan");
```

```
S.Mer=Kg; S.EdCena=1.99; S.GodnaDo=toDate;
```

2. При инициализиране :

```
Date NewYear={1,1,2020};
```

АТД СТРУКТУРА В C++: ОПЕРАЦИИ И РЕЛАЦИИ

- **Въвеждане и извеждане:** структурите могат да се въвеждат и извеждат само поелементно (ако >>/<< не са предефинирани за съответния тип)
- **Присвояване** — на променлива от тип структура може да бъде присвоена променлива от същия тип структура

```
Point p1, p2;  
p1.x=3;  
p1.y=2;  
p2=p1; /*копира полетата на p1 в p2*/
```

, което е еквивалентно на:

```
p2.x=p1.x;  
p2.y=p1.y;
```
- **Всички останали операции** със структури се извършват поелементно (могат да се предефинират от програмиста)
- **Релации** със структури не са дефинирани стандартно (програмистът може сам да предефинира избрана релация за някой тип структура)
- Структурите могат да се **подават като параметри на функции** и да бъдат техен **резултат**

АТД СТРУКТУРА В C++: ОСОБЕНОСТИ

- Когато не е необходимо името на типа структура може да се пропусне (анонимна структура, *anonymous*)

! Не е стандарт, зависи от компилатора и се позволява само за някои версии на езика.

- (1) когато всички необходими променливи се декларира на едно място

```
struct {float x, y;} complex;
```

Достъп: `complex.x`;

- (2) при вложени структури

```
struct somestruct{  
    struct // Анонимна  
        { int x, y; } point;  
    int type;  
} w; // Достъп w.point.y;
```

- MS C++ позволява последното поле на структура да бъде масив с недефиниран размер, което дава възможност към структурата да се добавят низове с различна дължина или друг масив

```
struct PERSON {  
    unsigned number;  
    char      name[];           //масив с недефиниран размер};
```

ПРИМЕРНА ПРОГРАМА ЗА РАБОТА С СТРУКТУРИ

Условие: Да се напише програма, която позволява да се въвеждат от клавиатурата данни за две книги и да се извеждат въведените данни

Проектиране:

(1) Представяне на данните:

Входни данни: Данните за всяка книга удобно се представят чрез комбинирания тип данни структура

(2) Алгорътъм:

Извеждането на данните за една книга може да се обособи в отделна функция

ПРОГРАМНА РЕАЛИЗАЦИЯ НА ПРИМЕРА

Програмата използва шаблонния клас `string` от заглавния файл `<cstring>` от STL:

```
#include <iostream>
#include <cstring>
using namespace std;
struct Book {
    char  title[50];
    char  author[50];
    char  subject[100];
    int   book_id;
};
int main( ) {
    struct Book Book1;           // деклариране на Book1 от тип Book
    struct Book Book2;           // деклариране на Book2 от тип Book

    // въвеждане на данни за Book 1
    strcpy( Book1.title, "Fundamentals of Programming C++");
    strcpy( Book1.author, „Richard Halterman");
    strcpy( Book1.subject, "C++ Programming");
    Book1.book_id = 345407;
```

```
// въвеждане на данни за Book 2
strcpy( Book2.title, "Data Structures and Algorithm in C++");
strcpy( Book2.author, "Clifford Shaffer");
strcpy( Book2.subject, " C++ Programming");
Book2.book_id = 6495700;

// извеждане на данните за Book1
printBook( Book1 );

// извеждане на данните за Book2
printBook( Book2 );

return 0;

}
```

```
void printBook( struct Book book ) {
    cout << "Book title : " << book.title <<endl;
    cout << "Book author : " << book.author <<endl;
    cout << "Book subject : " << book.subject <<endl;
    cout << "Book id : " << book.book_id <<endl;
}
```